COMS 4701 Exam 1 Master Cheat Sheet

Comprehensive Review, Examples, and Exam Pro-Tips

1. SEARCH SPACE FORMULATION

To formally define a search problem, map the environment to a structured mathematical model using 6 components:

- **State Representation:** A data structure (list, set, matrix) capturing all dynamic variables.
- **Initial State:** Starting configuration.
- **Possible States:** Total size of the state space.
- **Possible Actions:** Valid moves for the agent.
- **Transition Model:** Function returning the new state given a current state and action.
- **Goal Test:** Condition for winning.

Exam Tips: Formulation

- **Minimalism:** The state should ONLY include what changes (e.g., agent position, remaining food). Do NOT include static elements (like maze walls).
- **State Space Size:** Use combinatorics. If assigning N objects to M slots, it's often M^N.

Example: 3-Tower Hanoi Puzzle

Problem: 3 pegs (1,2,3) and 3 disks (S, M, L). Move all to the right peg. Cannot place a larger disk on a smaller one.
Representation: 3 ordered sets. e.g., $\{1 : \{M, L\}, 2 : \{\}, 3 : \{S\}\}$. Top disk is the first element.
Initial State: $\{1 : \{S, M, L\}, 2 : \{\}, 3 : \{\}\}$
Possible States: $3^3 = 27$ configurations.
Actions: Move top disk from one peg to another.
Goal Test: $\{1 : \{\}, 2 : \{\}, 3 : \{S, M, L\}\}$

2. UNINFORMED SEARCH STRATEGIES

Searches without heuristic guidance. Strict tie-breaking rules and an explored set (to prevent loops) are critical.

Breadth-First Search (BFS)

- **Structure:** Queue (FIFO). Explores layer by layer.
- **Completeness/Optimality:** Complete. Optimal if step costs are equal.

Depth-First Search (DFS)

- **Structure:** Stack (LIFO). Explores deepest first.
- **Completeness/Optimality:** Incomplete (can get stuck in infinite loops without explored sets). Not optimal.

Exam Tips: BFS & DFS Trace

- **Order of Visit $\neq$ Order of Generation:** Visit order is the exact order nodes are POPPED from the queue/stack, not when they are added.
- **Tie-Breaking:** Apply rules (e.g., U-R-D-L) strictly when pushing neighbors into the frontier.
- **DFS Trick:** When pushing to a stack, push in *reverse* of the priority order so the highest priority is on top.

Iterative Deepening Search (IDS)

Runs DFS repeatedly with increasing depth limits.

Example: Integer State Space

Problem: Start=1, Goal=11. Children of a are $2a$ and $2a + 1$.
L=0: (1)
L=1: $(1, 2, 3)$
L=2: $(1, 2, 4, 5, 3, 6, 7)$
L=3: $(1, 2, 4, 8, 9, 5, 10, 11) \leftarrow$ Stops at 11.

Exam Tips: IDS

- Always restart from the root node for each new limit L. Do not continue from the previous queue.

3. INFORMED (HEURISTIC) SEARCH

Definitions: $g(n)$ = cumulative actual cost from start.
$h(n)$ = estimated cost to goal.
$f(n)$ = total estimated path cost.

Uniform Cost Search (UCS)

Expands node with lowest $g(n)$. Priority Queue keyed by cost. If a node is reached with a **lower** cost, update its key and parent.

A* Search

Expands node with lowest $f(n) = g(n) + h(n)$.

Exam Tips: UCS & A*

- **The Update Trap:** If you find a new path to a node already in the PQ, calculate the new $g(n)$ or $f(n)$. If it's strictly LESS than the old value, update it AND change its parent.
- Keep a tabular trace (Node | g | h | f | Parent) on your scratch paper.

4. HEURISTICS EVALUATION

Let $h^*(n)$ be the true minimum cost from n to goal, and $c(A \to B)$ be the edge cost.

- **Admissibility (Never Overestimates):**
  $h(n) \leq h^*(n)$ for all nodes.
- **Consistency (Satisfies Triangle Inequality):**
  $h(A) - h(B) \leq c(A \to B)$ for all edges.
- *Rule:* Consistency $\implies$ Admissibility.

Exam Tips: Evaluating Heuristics

- **Step 1:** Before looking at the given h values, calculate the TRUE shortest path h^* for every node manually.
- **Step 2 (Admissibility):** Compare $h \leq h^*$. Any single violation ruins it.
- **Step 3 (Consistency):** Do subtraction $h(start) - h(end)$. If the drop is larger than the edge cost, it's inconsistent.

Example: Evaluating Heuristics

Given Graph: $S \to B$ (cost 1), $B \to G$ (cost 3), $S \to G$ (cost 5).
True Costs: $h^*(S) = 4$, $h^*(B) = 3$.
H1: $h(S) = 4, h(B) = 2$. Admissible (Yes, $4 \leq 4, 2 \leq 3$). Consistent (No, $4 - 2 \not\leq 1$).
H2: $h(S) = 6, h(B) = 3$. Admissible (No, $6 > 4$).
H3: $h(S) = 3, h(B) = 2$. Admissible (Yes). Consistent (Yes, $3 - 2 \leq 1$ and $2 - 0 \leq 3$).

5. CONSTRAINT SATISFACTION (CSPS)

AC-3 (Arc Consistency)

Filters domains before search. For arc $X \to Y$, if X has a value with no valid pairing in Y, remove it from X's domain. Re-queue neighbors if reduced.

Backtracking & Forward Checking

- **Pure Backtracking:** Assigns variables blindly, fails when a constraint with a prior assignment breaks.
- **Forward Checking (FC):** Assign a value, then immediately remove conflicting values from unassigned neighbors.
- **Domain Wipeout:** If FC leaves a neighbor with an empty domain $\{\}$, backtrack immediately.

Exam Tips: CSP Search Trees

- **Pure Backtracking:** Act dumb. Try values in exact numerical/lexical order. Don't skip a bad value using human intuition; write it down and mark "FAIL".
- **Forward Checking:** The moment ANY unassigned variable's domain hits $\{\}$, stop expanding the current node and backtrack immediately.

6. SIMULATED ANNEALING (LOCAL SEARCH)

Algorithm to find global optimum while escaping local optima by occasionally accepting worse moves.

- **Difference:** $\Delta h = \text{cost}_{new} - \text{cost}_{current}$.
- **If Improving ($\Delta h \leq 0$):** Always accept.
- **If Worsening ($\Delta h > 0$):** Accept with probability $P(\text{accept}) = \exp(-\Delta h / T)$.
- **Cooling:** $T_{new} = \alpha \times T_{old}$ (e.g., $\alpha = 0.9$).

Exam Tips: Simulated Annealing

- **Min vs Max:** Carefully read if the problem is Minimization (like TSP distance) or Maximization (like profit). Adjust the sign logic of Δh accordingly. "Improving" means negative Δ for distance, positive for profit.
- **Temperature T:** High T (early) = random walk, high acceptance of bad moves. Low T (late) = greedy hill-climbing.

[10pt,twocolumn]article [margin=0.5in]geometry amsmath, amssymb enumitem titlesec

COMS 4701 AI Midterm Cheat Sheet (HW2)

1. Heuristics & A^* Search

Admissibility (): A heuristic $h(n)$ is admissible if it never overestimates the true optimal cost $h^*(n)$ to the goal.
- Formula: $0 \leq h(n) \leq h^*(n)$ for all nodes n.
- Guarantees A^* will find the optimal path in tree search.

Dominance (): Heuristic h_2 dominates h_1 if for all nodes n, $h_2(n) \geq h_1(n)$ (and is strictly greater for at least one node).
- A^* using the dominant heuristic (h_2) will expand **fewer or equal** nodes than A^* using h_1.
- **Derived Heuristic:** If h_1 and h_2 are admissible, then $h_3(n) = \max(h_1(n), h_2(n))$ is also admissible and dominates both.

Consistency / Monotonicity (): A heuristic is consistent if the estimated cost from n to the goal is no greater than the step cost to a successor n' plus the estimated cost from n' to the goal.
- Formula: $h(n) \leq c(n, n') + h(n')$
- Consistency $\implies$ Admissibility.
- Guarantees A^* graph search is optimal (first time a state is expanded, the optimal path to it has been found).

2. Local Search Algorithms

Hill Climbing ():
- Greedy local search: always moves to the neighbor with the highest value (or lowest cost).
- **Flaw:** Highly sensitive to the initial state; easily gets trapped in **Local Maxima** (valleys) and cannot backtrack.

Local Beam Search ():
- Maintains k states instead of 1.
- At each step, generates all neighbors of all k states, pools them together, and selects the top k to form the next beam.
- **Benefit:** Threads share information. A sub-optimal node can act as a bridge to the global maximum.
- **Beam Concentration:** Risk of all k states clustering in the same local optimum, losing diversity (behaves like a single hill-climber).

Stochastic Beam Search:
- Selects the next k states probabilistically to maintain diversity and prevent concentration.
- Probability formula: $P(n) = \frac{V(n)}{\sum_{i \in pool} V(i)}$

3. Simulated Annealing (SA)

Escapes local optima by allowing "bad moves" (cost increases) with a certain probability, governed by a "Temperature" (T).

Acceptance Probability:
- If $\Delta E \leq 0$ (improvement), always accept.
- If $\Delta E > 0$ (worse state), accept with probability: $P = e^{-\Delta E/T}$

Temperature (T) Dynamics:
- **High T (Exploration):** $P \approx 1$. Accepts worse moves easily, exploring the global state space broadly.
- **Low T (Exploitation):** $P \approx 0$. Rejects worse moves, behaving purely like greedy Hill Climbing to fine-tune the solution.

Cooling Schedules:
- **Logarithmic** ($T \propto 1/\ln(t)$): Cools extremely slowly. Provides a theoretical guarantee of finding the global optimum (Geman-Geman theorem).
- **Exponential** ($T \propto \alpha^t$): Cools very fast. Risks "freezing" into a sub-optimal local minimum early on.
- **Linear** ($T \propto -t$): Steady decrease, hits exactly 0 at a predetermined iteration.

4. Constraint Satisfaction Problems (CSP)

Defined by Variables (X), Domains (D), and Constraints (C).

Forward Checking (FC):

- **Trigger:** When variable X is assigned a value.
- **Action:** Look at all *unassigned* variables Y connected to X by a constraint. Remove any values from D_Y that conflict with X's new assignment.
- **Limitation:** Highly localized. It does not check for future conflicts between two unassigned variables.

Arc Consistency (AC-3):

- A directed arc $X \to Y$ is consistent if, for every value in D_X, there is some allowed value in D_Y.
- **Algorithm:** Maintains a queue of arcs. If making $X \to Y$ consistent causes D_X to shrink, **all arcs pointing to** X (e.g., $Z \to X$) must be re-added to the queue and re-checked.
- **Advantage:** Propagates constraints globally. Can trigger a "cascade" effect that drastically reduces the search space (or even solves the puzzle entirely without search).

[10pt,a4paper,twocolumn]article [margin=0.5in]geometry amsmath enumitem titlesec xcolor

COMS 4701: Artificial Intelligence – Mid-term Cheat Sheet

7. What is AI?

AI aims to design **Intelligent Agents** that perceive environments and act to maximize success. There are 4 main schools of thought:

- **Thinking humanly**: Cognitive approach (e.g., modeling internal brain activities).
- **Acting humanly**: Turing Test (1950) – pass if it fools a human interrogator.
- **Thinking rationally**: Laws of thought/logic (can be computationally heavy; not all knowledge can be expressed this way).
- **Acting rationally**: **Our approach!** Acting to achieve the best expected outcome (maximizing goal achievement given available info).

8. Rational Agents

An agent perceives its environment via **sensors** and acts via **actuators**.

- **Core Formula**: $Agent = Architecture + Program$
- **Rationality**: Judged based on (1) Performance measure, (2) Prior knowledge of the environment, (3) Possible actions, (4) Percept sequence to date.

PEAS Specification (Defining the Task Environment):

- **P**erformance: Criteria for success.
- **E**nvironment: The setting/world.
- **A**ctuators: Mechanisms to act.
- **S**ensors: Mechanisms to perceive.

Ex (Self-Driving Car): **P** (safety, time, legal, comfort), **E** (roads, pedestrians), **A** (steering, brakes, horn), **S** (cameras, GPS, sonar).

9. Environment Types

- **Fully vs. Partially Observable**: Can sensors access the *complete* state of the environment at all times?
- **Deterministic vs. Stochastic**: Is the next state completely determined by the current state + agent's action? (If exceptions are only due to other agents, it's *Strategic*). Stochastic models uncertainty with probabilities.
- **Static vs. Dynamic**: Does the environment change while the agent is thinking? (*Semi-dynamic*: environment doesn't change, but the agent's performance score does).
- **Discrete vs. Continuous**: Limited, clearly defined percepts/actions (e.g., Chess) vs. infinite/fluid (e.g., Driving).
- **Single vs. Multi-agent**: Is the agent alone, or are there others (cooperative/competitive)?
- **Known vs. Unknown**: Does the designer/agent know the "laws of physics" (rules) of the environment?

Ex: **Chess** (Fully, Multi, Deterministic, Static, Discrete). **Roomba** (Partially, Single, Stochastic, Dynamic, Continuous).

10. Agent Types (Increasing Generality)

1. **Simple Reflex Agents**: Select action based *only* on the current state (ignores percept history).
2. **Model-based Agents**: Maintain an internal model of how the world evolves independently and how the agent's actions affect it.
3. **Goal-based Agents**: Combine environment model with goal info to choose actions that achieve the goal.
4. **Utility-based Agents**: Use a utility function to measure "happiness" or performance, making decisions to maximize the expected outcome.

11. State Representations

Increasing expressive power:

- **Atomic**: State is a black box with no internal structure. *Algorithms*: Search, Games, MDP, HMM.
- **Factored**: State has explicit attribute-value properties (e.g., GPS location, gas level). *Algorithms*: CSPs, Bayesian Networks.
- **Structured**: Explicit relationships between objects in a state. *Algorithms*: First-Order Logic, NLP.

12. Problem Solving & Search

- **Search Agents**: Find a sequence of actions leading to a goal. *Uninformed* (no domain knowledge, e.g., BFS/DFS/UCS) vs. *Informed* (uses heuristics to reach goals faster, e.g., A^*, Greedy).
- **Adversarial Search (Games)**: Involves uncontrollable opponents. The optimal solution is a *strategy/policy*, not a single sequence (e.g., If opponent does A, agent does B). *Methods*: Minimax, $\alpha - \beta$ pruning.
- **Constraint Satisfaction (CSP)**: Care about the goal state itself, not the path. All paths are the same depth. Find assignments to variables that satisfy all constraints (e.g., Sudoku grids must contain 1-9, 8-Queens).

[10pt,landscape]article [utf8]inputenc multicol [margin=0.5in]geometry amsmath, amsfonts, amssymb xcolor enumitem titlesec

COMS 4701: Artificial Intelligence - Search Agents Mid-term Cheat Sheet

Columbia University — Topics: Goal-Based Agents, Problem Formulation, and Search Strategies

13 Agent Types

- **Reflex Agents:** Act based on a direct mapping from the current state to an action.
- **Goal-Based Agents:** Problem-solving or planning agents. They consider the future impact of actions to find a sequence that reaches a goal.

14 Problem Formulation

A search problem is formally defined by:

- **Initial State:** The starting point for the agent.
- **State Space:** The set of all states reachable from the initial state by any sequence of actions.
- **Action Space:** $Actions(s)$ returns the set of executable actions in state s.
- **Transition Model:** $Results(s, a)$ returns the state resulting from action a in state s.
- **Goal Test:** A condition that determines if a state is a goal.
- **Path Cost:** A function assigning a numeric cost to a path, usually based on a performance measure.

15 The Search Process

- **State Space:** Physical configurations of the problem.
- **Search Space:** Abstract tree/graph of possible solutions.
- **Search Tree Nodes:** Contain state, parent, actions, depth, and path cost.
- **Frontier (Fringe):** The set of all leaf nodes available for expansion.
- **Expansion:** Applying all valid actions to a node to generate its children.

16 Search Space Regions

Search divides the space into three sets:

1. **Explored (Closed List):** Nodes already visited and expanded.
2. **Frontier (Open List):** Nodes discovered but not yet expanded.
3. **Unexplored:** States not yet reached by the search.

17 Core Algorithms

17.1 Tree Search

- Initialize frontier with initial state.
- While frontier is not empty:
- Remove a node; if it passes the goal test, return success.
- Expand the node and add neighbors to the frontier.
- **Risk:** Can get stuck in infinite loops due to repeated states.

17.2 Graph Search

- Same as Tree Search but adds an **Explored Set**.
- Only add a neighbor to the frontier if it has not been visited before (not in frontier or explored set).
- Prevents redundant work and infinite loops in cyclic graphs.

18 Strategy Evaluation

- **Completeness:** Guaranteed to find a solution if one exists.
- **Optimality:** Guaranteed to find the least-cost solution.

- **Time Complexity:** Number of nodes generated.
- **Space Complexity:** Maximum nodes stored in memory.

Variables for Complexity:

- b: Maximum branching factor (actions per state).
- d: Depth of the shallowest goal.
- m: Maximum depth of the state space (may be ∞).

19 Classic Problem Examples

- **8-Queen:** Place 8 queens so none attack. Solving requires searching through roughly 1.8×10^{14} sequences.
- **8-Puzzle:** 3×3 grid with 8 numbered tiles and a blank space. Actions are moving the blank up, down, left, right.
- **Route Finding:** Navigating between cities. States are $In(City)$; Actions are $Go(Neighbor)$.

20 Real-World Applications

- **TSP:** Shortest tour visiting every city exactly once.
- **VLSI Layout:** Positioning components on a microchip to minimize area and wire length.
- **Protein Design:** Searching for amino acid sequences that fold into specific 3D shapes.
- **Robot Navigation:** Pathfinding in continuous 2D/3D spaces.

[10pt,landscape]article [margin=0.5in]geometry multicol amsmath amssymb enumitem

COMS 4701: Uninformed Search Agents Cheat Sheet

Uninformed Search Overview

Definition: Search algorithms that use no domain knowledge beyond the initial state, the actions, and the goal test (also known as brute force search).
Strategies: BFS, DFS, DLS, IDS, UCS.

1. Breadth-First Search (BFS)

- **Concept:** Expand the shallowest node first. Searches layer by layer with each layer organized by node depth.
- **Implementation:** Frontier is a FIFO Queue.
- **Complete:** Yes (if branching factor b is finite).
- **Time Complexity:** $O(b^d)$ where d is the depth of the shallowest solution. $(1 + b + b^2 + ... + b^d)$.
- **Space Complexity:** $O(b^d)$. Memory requirement and exponential time complexity are the biggest handicaps.
- **Optimal:** Yes (if step cost is 1).

2. Depth-First Search (DFS)

- **Concept:** Expand the deepest nodes first. Searches branch by branch, pursued to maximum depth.
- **Implementation:** Frontier is a LIFO Stack.
- **Complete:** No (fails in infinite-depth spaces). Complete in finite spaces.
- **Time Complexity:** $O(b^m)$ where m is max depth. Can be bad if $m \gg d$; but if solutions are dense, may be much faster than BFS.
- **Space Complexity:** $O(bm)$. Linear space complexity! (Needs to store only a single path from the root to a leaf node, along with the remaining unexpanded sibling nodes).
- **Optimal:** No.

3. Depth-Limited Search (DLS)

- **Concept:** DFS with depth limit L. Nodes at level L have no successors. Use when there is prior knowledge about the problem space.
- **Complete:** No.
- **Time Complexity:** $O(b^L)$.
- **Space Complexity:** $O(bL)$.
- **Optimal:** No.

4. Iterative-Deepening Search (IDS)

- **Concept:** Combines benefits of BFS and DFS. Iteratively increases the search limit until the depth of the shallowest solution d is reached. Applies DLS with increasing limits.
- **Complete:** Yes.
- **Time Complexity:** $O(b^d)$. $(d+1)b^0 + db^1 + ... + b^d$.
- **Space Complexity:** $O(bd)$. Same as DFS for level d.
- **Optimal:** Yes (if step cost is 1).
- **Theorem (Korf 1985):** IDS is asymptotically optimal among brute force tree search in terms of time, space, and length. Wasted computation is negligible for large spaces (bounded by $C = (1 - 1/b)^{-2}$).

5. Uniform-Cost Search (UCS)

- **Concept:** Prioritize by cost, not depth. Expand node n with the lowest path cost $g(n)$. Searches layer by layer organized by path cost.
- **Implementation:** Frontier is a Priority Queue (Heap) ordered by $g(n)$.
- **Complete:** Yes (if solution has a finite cost).
- **Time Complexity:** $O(b^{C^*/\epsilon})$, where C^* is optimal solution cost and ϵ is min step cost. Effective depth is C^*/ϵ.
- **Space Complexity:** $O(b^{C^*/\epsilon})$.
- **Optimal:** Yes. Explores the space in every direction because no information is provided about the goal.

Frontiers & Examples

- **BFS Frontier:** Consists of nodes of similar depth (horizontal layer).
- **DFS Frontier:** Consists of unexplored siblings of all ancestors. Exhausts one branch at a time.
- **UCS Frontier:** Consists of nodes of various depths (jagged). Expands lowest-cost nodes.
- **Example Algorithm Behaviors (Start $S \to$ Goal G):**
  - **BFS:** Enqueues neighbors. Queue: $S \to A, B, C$. Visit order is layer by layer.
  - **DFS:** Pushes to stack. Stack: $S \to C, B, A$. Visits $S \to A \to$ children of A before returning to B.
  - **UCS:** Always pops the node with minimum $g(n)$. E.g., if path $S \to C$ costs 2, and $S \to A$ costs 3, it pops and expands C first regardless of depth.

[10pt,landscape]article multicol geometry amsmath, amssymb enumitem
top=1cm,left=1cm,right=1cm,bottom=1cm

Informed Search & Heuristics

Blind searches (BFS, DFS) are combinatorial and intractable (exponential space/time). Informed search reduces blindness by using **intuition/domain knowledge** called **Heuristics**.

- **Heuristic Function ($h(n)$)**: Estimates the cost from node n to the closest goal.
- Does not have to be perfect, just guides the search to make the problem tractable.

Greedy Best-First Search

Expands the node that appears to be closest to the goal.

- **Evaluation Function**: $f(n) = h(n)$
- **Example**: Using straight-line distance ($h_{SLD}(n)$) from city n to the destination.
- **Implementation**: Uses a Priority Queue (Min-Heap) sorted by $h(n)$.

A* Search

Minimizes the total estimated solution cost.

- **Cost of path to n**: $g(n)$
- **Estimated cost n to goal**: $h(n)$
- **Evaluation Function**: $f(n) = g(n) + h(n)$
- $f(n)$ is the estimated cost of the cheapest solution through n.
- **Properties**: Complete (yes), Optimal (yes). Time/Space complexity ranges from polynomial (with an excellent heuristic) to exponential $O(b^d)$ (with a poor heuristic).

Admissible Heuristics

A good heuristic must be **admissible** (optimistic). It never overestimates the true cost to reach the goal.

- **Condition**: $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost from n to the goal.
- **Theorem 1**: If $h(n)$ is admissible, A^* using **tree search** (states do not repeat, e.g., Tic-Tac-Toe) is **optimal**.

Proof of Optimality (Tree Search)

Suppose G_o is the optimal goal, G_s is a suboptimal goal. Let n be an unexpanded node on the shortest path to G_o.

1. $f(G_s) = g(G_s)$ and $f(G_o) = g(G_o)$ (since $h = 0$ at goals).
2. G_s is suboptimal $\implies f(G_s) > f(G_o)$.
3. $h(n) \leq h^*(n)$ (admissibility).
4. $g(n) + h(n) \leq g(n) + h^*(n) = g(G_o)$.
5. $f(n) \leq f(G_o)$.
6. Therefore, $f(G_s) > f(n)$. A^* will expand n before G_s, never selecting the suboptimal goal.

Consistent (Monotonic) Heuristics

For **graph search** (where states can repeat, e.g., pathfinding, 8-puzzle), we need a stronger condition: **Consistency**.

- **Condition**: $h(n) \leq c(n, a, n') + h(n')$, where c is the step cost from n to n'.
- Preserves the **triangular inequality**.
- **Property**: $f(n)$ is monotonically non-decreasing along any path. $f(n') \geq f(n)$.
- **Theorem 2**: A^* using a consistent heuristic is **optimal for graph search**.
- *Lemma*: Every consistent heuristic is admissible (but not vice versa).

Proof of Optimality (Graph Search)

When A^* selects a node n for expansion, the optimal path to that node has been found.

1. Assume A^* reaches n' sub-optimally. There is an optimal path through n not yet found.
2. $f(n') \leq f(n)$ (since n' expanded first).
3. $g(n') + h(n') \leq g(n) + h(n) \implies g(n') - g(n) \leq h(n) - h(n')$.
4. Sub-optimal path means: $g(n') > g(n) + c(n, a, n')$.
5. Therefore: $c(n, a, n') < g(n') - g(n) \leq h(n) - h(n')$.
6. $c(n, a, n') < h(n) - h(n') \implies h(n) > c(n, a, n') + h(n')$.
7. This contradicts the consistency definition.

Dominance & Relaxed Problems

- **Dominance**: If $\forall n, h_1(n) \leq h_2(n)$, then h_2 **dominates** h_1. A^* using h_2 will expand fewer nodes than h_1 and is more efficient. Tradeoff: computational cost vs. accuracy.
- **Relaxed Problems**: Good heuristics are crafted by relaxing problem constraints.
- **8-Puzzle Example**:
  - $h_1(n)$: Number of misplaced tiles (assumes a tile can move anywhere).
  - $h_2(n)$: Total Manhattan distance (assumes a tile can move to any adjacent square, ignoring occupied squares).
  - h_2 dominates h_1. Both are admissible.

Search Comparisons Summary

Iterate on node depth:

- BFS: searches layers of increasing depth.
- IDS: searches subtrees of increasing depth.

Iterate on cost/heuristic:

- UCS: searches layers of increasing $g(n)$.
- Greedy: searches layers of increasing $h(n)$.
- A^*: searches layers of increasing $f(n) = g(n) + h(n)$.
- IDA^*: searches subtrees of increasing path cost + heuristic.

AI Search Agents: Local Search Cheat Sheet

[10pt,twocolumn]article [margin=0.5in]geometry amsmath amssymb enumitem

1. Motivation & Definition

- **Systematic search fails when:** State spaces are massive (10^{10} to 10^{100}), memory is constrained, or the path to the solution is irrelevant (optimization problems).
- **Local Search Paradigm:** Iterative improvement algorithms that maintain a single "current" state and attempt to improve it. Move only to neighbors.
- **Advantages:** Uses $O(1)$ memory; handles continuous/massive discrete spaces.
- **Applications:** Pure optimization (maximize/minimize $f : \mathcal{S} \to \mathbb{R}$), Constraint Satisfaction Problems (e.g., N-queens, scheduling).

2. State-Space Landscape

Geometric metaphor mapping configurations (x-axis) to objective function values $f(s)$ (y-axis).

- **Global optimum:** $s^* = \arg\max_{s \in \mathcal{S}} f(s)$. Target state.
- **Local maximum:** $f(s) \geq f(s')$ for all neighbors $s' \in \mathcal{N}(s)$. (Algorithm trap).
- **Plateau:** $f(s) = f(s')$ for all $s' \in \mathcal{N}(s)$. Causes stagnation.
- **Shoulder:** A flat region that has a higher region adjacent to it.

3. Hill Climbing (Steepest Ascent)

Greedy search: Always move to the best immediate neighbor. "Trying to find the top of Mount Everest in a thick fog while suffering from amnesia."
- **Selection:** $next = \arg\max_{s' \in \mathcal{N}(s)} f(s')$.
- **Properties:** Terminates at local optima. Monotonic ($f(s_{t+1}) \geq f(s_t)$). Not complete, not optimal.
- **Complexity:** Time is $O(k \cdot |\mathcal{N}(s)|)$ where k is steps to local optimum. Space is $O(1)$.

Variants:
- **Sideways moves:** Allow $f(next) = f(current)$ to escape plateaus (limit max moves to avoid infinite loops).
- **Random-restart:** Run multiple trials from random initial states. Returns best solution across all trials (e.g., 99.99% success for 8-Queens with ≈ 6 restarts).
- **Stochastic:** Randomly select among improving moves (probability proportional to improvement).

Example: 8-Queens Problem
- *State space:* $8^8 \approx 1.67 \times 10^7$ states. Neighbors: 56.
- *Objective:* Minimize attacking pairs (Steepest Descent).
- *Result:* Fast convergence (~ 4 steps) but frequently suboptimal (success rate $\sim 14\%$ from random starts).

4. Local Beam Search

Maintains k states simultaneously (shares information between parallel searches).
1. Start with k random states.
2. Generate all successors of all k states.
3. Select the k best successors from the combined pool.
4. Repeat until goal found or convergence.

Stochastic beam search: Selects k successors probabilistically to maintain diversity and prevent premature convergence.

5. Genetic Algorithms (GA)

Population-based stochastic optimization inspired by biological natural selection.
- **Components:** Population (k individuals), Genotype (string representation), Fitness function f : Genotype $\to \mathbb{R}^+$.
- **Selection:** Fitness-proportionate ($P(select\ i) = f(s_i)/\sum_j f(s_j)$), Tournament (best from random subset), Rank-based.
- **Crossover:** Single-point, Uniform, Order-preserving (probability $p_c \in [0.6, 0.9]$).
- **Mutation:** Bit-flip, Swap (probability $p_m \in [0.001, 0.01]$).

GA Example: 8-Queens
- *Genotype:* String of digits $\{1, \ldots, 8\}$. Position = column, Value = row. Example: [3, 2, 7, 4, 8, 5, 5, 2].
- *Fitness Function:* $f(s) = \binom{8}{2} - conflicts(s) = 28 - conflicts(s)$. Perfect solution $f = 28$.

6. Simulated Annealing (SA)

Inspired by metallurgy (controlled cooling to reach low-energy crystalline structure). Allows worse moves probabilistically to escape local optima.
- **Metropolis Criterion:**

$$P(accept) = \begin{cases} 1 & \text{if } \Delta E \leq 0 \\ e^{-\Delta E/T} & \text{if } \Delta E > 0 \end{cases}$$

  where E is objective function (cost) and T is Temperature.
- **Cooling Schedules $T(t)$:**
  - *Logarithmic:* $T(t) = c/\log(1+t)$. Guarantees global optimum but impractically slow.
  - *Exponential:* $T(t) = T_0 \alpha^t$. Most practical ($\alpha \in [0.8, 0.99]$).
  - *Linear:* $T(t) = T_0 - \beta t$. Very fast, quick approximation.

- **Algorithm Logic:** High $T \to$ accept many worse moves (exploration). Low $T \to$ highly selective (exploitation).

Example: Traveling Salesman Problem (TSP)

- *Problem:* Shortest route visiting n cities exactly once. State space: $(n-1)!/2$.
- *Complexity:* Intractable for $n \geq 20$ via exhaustive search.
- *Neighborhoods:* 2-opt (swap 2 edges), 3-opt, Or-opt.
- *Results:* SA typically finds a tour within 2-5% of optimal efficiently.

7. Algorithm Comparison

- **Hill Climbing:** $O(1)$ memory, local optimality, fast. Best for smooth landscapes.
- **Random Restart:** $O(1)$ memory, high probability of global optimality, medium speed. Best for multiple local optima.
- **Beam Search:** $O(k)$ memory, local optimality, medium speed. Best for parallel exploration.
- **Genetic Algorithm:** $O(n)$ memory, global search, slow. Best for complex landscapes.
- **Simulated Annealing:** $O(1)$ memory, global w/ guarantees (theoretical), medium speed. Best for general optimization.

[10pt,twocolumn]article [margin=0.5in]geometry amsmath enumitem

Artificial Intelligence: Constraint Satisfaction Problems (CSPs)

1. CSP Definition

A CSP is a type of search problem where we care about the goal itself, not just the path[cite: 881, 882]. It consists of three main elements:

- **Variables:** A set $X = \{X_1, X_2, ..., X_n\}$[cite: 909].
- **Domains:** A set $D = \{D_1, D_2, ..., D_n\}$ for each variable[cite: 910].
- **Constraints:** A set C specifying allowable combinations of values[cite: 911].

Goal: Find a consistent assignment (a solution) that satisfies all constraints[cite: 917, 918].

2. Varieties of Variables & Constraints

Variables:

- *Discrete (Finite):* e.g., Map coloring, 8-queens. $O(d^n)$ complete assignments for n variables and d values[cite: 1046, 1047, 1048, 1049].
- *Discrete (Infinite):* Integers/strings. Requires a constraint language (e.g., job scheduling $T_1 + d \leq T_2$)[cite: 1050, 1051, 1052].
- *Continuous:* Common in operations research (e.g., Hubble Telescope scheduling, linear programming)[cite: 1053, 1054, 1055, 1056].

Constraints:

- *Unary:* Involves a single variable (e.g., $SA \neq green$)[cite: 1061].
- *Binary:* Involves pairs of variables (e.g., $SA \neq WA$). Represented as arcs in a constraint graph[cite: 1026, 1062].
- *Global:* Involves 3+ variables (e.g., `Alldiff` in Sudoku or Cryptarithmetic)[cite: 1063].
- *Preferences (Soft Constraints):* Represented by costs; constrained optimization problems[cite: 1064, 1066, 1067].

3. Key Examples

- **Map Coloring:** Variables are regions (WA, NT, SA, etc.); Domains are colors (red, green, blue); Constraints are adjacent regions must have different colors[cite: 942, 943, 967].
- **8-Queens:** Variables $Q_1 ... Q_8$ for each queen. Domain 1-8 (columns). Constraint: no queen attacks another[cite: 1074, 1087, 1088].
- **Cryptarithmetic (TWO+TWO=FOUR):** Variables are letters (T, W, O, F, U, R) and carry-overs (C_1, C_2, C_3). Domain 0-9. Constraints include `Alldiff(F,T,U,W,R,O)` and addition logic like $O + O = R + 10 * C_1$[cite: 1142, 1143, 1145, 1147].
- **Sudoku:** 81 variables (squares). Domain 1-9. 27 `Alldiff` constraints across rows, columns, and 3x3 boxes[cite: 1341, 1365, 1366].

4. Solving CSPs: Search & Heuristics

Backtracking Search (BTS): Basic uninformed depth-first search. Assigns one variable at a time and checks constraints on the go to avoid conflicts[cite: 1184, 1185, 1186]. Time complexity is $O(d^n)$[cite: 1597].

Heuristics to Improve BTS:

- **Minimum Remaining Values (MRV):** "Pick the hardest!" Chooses the variable with the fewest legal values left in its domain[cite: 1217, 1218].

- **Least Constraining Value (LCV):** "Pick the ones likely to work!" For a chosen variable, select the value that rules out the fewest values in remaining unassigned variables[cite: 1228, 1231].
- **Forward Checking (FC):** Detects inevitable failure early. Keeps track of remaining legal values for unassigned variables and terminates if any variable reaches zero legal values[cite: 1241, 1242]. *Limitation:* Does not check interactions between unassigned variables[cite: 1443].

5. Constraint Propagation (Inference)

Uses constraints to reduce the number of legal values, intertwining with or preceding search[cite: 1178, 1179, 1180].

- **Node Consistency:** All values in a variable's domain satisfy unary constraints[cite: 1460].
- **Arc Consistency (AC):** $X \to Y$ is consistent iff for every value $x \in X$, there is some allowed $y \in Y$[cite: 1466, 1475].
- **AC-3 Algorithm:** Makes a binary CSP arc-consistent. Adds arcs to a queue, removes them to check `REVISE(X, Y)`. If X's domain is reduced, adds all of X's neighbors back to the queue. Overall complexity: $O(n^2d^3)$[cite: 1542, 1547, 1549, 1551, 1567].

6. Problem Structure

Leveraging graph structure speeds up search[cite: 1043, 1590].

- **Independent Subproblems:** e.g., Tasmania in map coloring. Decomposing into subproblems of size c reduces complexity from $O(d^n)$ to $O((n/c)d^c)$[cite: 1591, 1598, 1601].
- **Tree-Structured CSPs:** Any two variables connected by only one path. Can be solved in linear time $O(nd^2)$[cite: 1616, 1659, 1660].
  1. Pick a root and Topological Sort (parents before children)[cite: 1627, 1628].
  2. Make Directed Arc Consistent (DAC) from $j = n$ down to 2[cite: 1618, 1633, 1634].
  3. Assign values from $i = 1$ to n without needing to backtrack![cite: 1636, 1637, 1645].
- **Nearly Tree-Structured:** Assign a fixed value to a variable (e.g., SA) to prune neighbor domains, turning the remaining graph into a tree[cite: 1675, 1676, 1677].